

08 — Orchestration Service: Advanced Modules (Deep Dive)

Module purpose: the remaining orchestration building blocks — managed RAG, guaranteed-JSON output, multi-turn memory — plus how sheet 07's pieces combine into one runnable pipeline and agent.

File 1 — document_grounding_module.py

```
from gen_ai_hub.orchestration.models.document_grounding import DocumentGrounding,
↳ DocumentGroundingFilter, GroundingModule

def build_grounding(input_placeholder="groundingRequest",
↳ output_placeholder="groundingOutput", data_repositories=None) -> GroundingModule:
    return GroundingModule(
        input_params=[input_placeholder],      # placeholder holding the query orchestration
↳ searches with
        output_param=output_placeholder,      # placeholder that gets filled with retrieved
↳ context
        filters=[DocumentGroundingFilter(id="default_filter",
↳ data_repositories=data_repositories or ["*"])],
    )

# Template must reference BOTH placeholders so the model sees question + retrieved context:
template = Template(messages=[
    SystemMessage("Answer using ONLY the provided context. If missing, say you don't know."),
    UserMessage("Question: {{?groundingRequest}}\nContext: {{?groundingOutput}}"),
])
config = OrchestrationConfig(template=template, llm=LLM(name="gpt-4o-mini"),
↳ grounding=DocumentGrounding(module_config=build_grounding()))
```

Working explanation: this is orchestration's **built-in RAG** — compare directly with sheet 05's hand-rolled rag_pipeline.py. There, YOU call similarity_search and inject context into the prompt yourself in Python. Here, AI Core does retrieval **inside the pipeline** — you just tell it which placeholder holds the query (input_params) and which placeholder to fill with results (output_param); the Template references both by name. Choose managed grounding for standard document Q&A with less code to maintain; choose the manual sheet-05 pipeline when you need custom retrieval logic (hybrid search, re-ranking, joining retrieval with live SQL data).

File 2 — structured_output_module.py

```
from gen_ai_hub.orchestration.models.response_format import ResponseFormatJsonSchema

TICKET_SCHEMA = {
    "type": "object",
    "properties": {
        "category": {"type": "string", "enum": ["BUG", "FEATURE_REQUEST", "QUESTION",
↳ "COMPLAINT"]},
        "urgency": {"type": "string", "enum": ["LOW", "MEDIUM", "HIGH", "CRITICAL"]},
        "summary": {"type": "string"},
    },
    "required": ["category", "urgency", "summary"], "additionalProperties": False, # no
↳ surprise extra keys
}
```

```
def build_structured_template(schema, schema_name="extraction_result") -> Template:
    return Template(
        messages=[SystemMessage("Extract structured info. Respond ONLY with JSON matching the
↪ schema."), UserMessage("{{?ticket_text}}")],
        response_format=ResponseFormatJsonSchema(name=schema_name, schema=schema,
↪ strict=True),
    )

# result content is now safe to parse directly:
parsed = json.loads(result.orchestration_result.choices[0].message.content)
```

Working explanation: `strict: true` **constrains token generation itself** so the model literally cannot emit a response that fails schema validation — this is fundamentally different (and more reliable) than a prompt instruction like “please respond in JSON,” which the model can still drift from (extra prose, wrong types, missing fields). `additionalProperties: false` closes the schema so no unexpected extra keys sneak in — a hard contract downstream code can rely on.

File 3 — conversation_history_module.py

```
class ConversationBuffer:
    def __init__(self, max_turns=10):
        self.max_turns, self.history = max_turns, []

    def add_user(self, text): self.history.append(UserMessage(text)); self._trim()
    def add_assistant(self, text): self.history.append(AssistantMessage(text)); self._trim()
    def _trim(self):
        max_messages = self.max_turns * 2
        if len(self.history) > max_messages: self.history = self.history[-max_messages:] #
↪ sliding window

def ask(orchestration_service, buffer, system_prompt, user_text) -> str:
    template = Template(messages=[SystemMessage(system_prompt), UserMessage("{{?text}}")])
    config = OrchestrationConfig(template=template, llm=LLM(name="gpt-4o-mini"))
    result = orchestration_service.run(config=config,
↪ template_values=[TemplateValue(name="text", value=user_text)],
        messages_history=buffer.history) # <- resent EVERY
↪ call
    answer = result.orchestration_result.choices[0].message.content
    buffer.add_user(user_text); buffer.add_assistant(answer)
    return answer
```

Working explanation: the Orchestration Service has **zero built-in session memory** — every `.run()` is completely independent server-side. `messages_history` is how the CLIENT fakes memory: rebuild and resend the whole (trimmed) conversation on every call. This is conceptually identical to sheet 05’s ChatMemory (LangChain-based) — same problem, same sliding-window solution, different implementation (hand-rolled list vs. LangChain’s ConversationBufferWindowMemory).

File 4 — orchestration_pipeline.py (full config, all modules wired)

```

def build_full_config(system_prompt, use_grounding=False, use_masking=True,
    ↪ use_filtering=True, model="gpt-4o-mini") -> OrchestrationConfig:
    user_message = "Question: {{?text}}\nContext: {{?groundingOutput}}" if use_grounding
    ↪ else "{{?text}}"
    template = Template(messages=[SystemMessage(system_prompt), UserMessage(user_message)])
    kwargs = {"template": template, "llm": build_llm(model=model)}
    if use_masking: kwargs["data_masking"] =
        ↪ build_masking(method=MaskingMethod.PSEUDONYMIZATION)
    if use_filtering: kwargs["filtering"] = build_filtering(input_filter=STRICT_FILTER,
        ↪ output_filter=STRICT_FILTER)
    if use_grounding: kwargs["grounding"] =
        ↪ DocumentGrounding(module_config=build_grounding(input_placeholder="text"))
    return OrchestrationConfig(**kwargs)

def run_pipeline(orchestration_service, config, user_text) -> dict:
    try:
        result = orchestration_service.run(config=config,
    ↪ template_values=[TemplateValue(name="text", value=user_text)])
        return {"ok": True, "answer": result.orchestration_result.choices[0].message.content}
    except Exception as e:
        return {"ok": False, "answer": None, "error": str(e)} # graceful degrade – a filter
        ↪ block is expected, not a bug

```

Working explanation: `build_full_config` is a **composable builder** — each module (masking, filtering, grounding) is added to `kwargs` only if its flag is on, then unpacked into `OrchestrationConfig(**kwargs)` once. `run_pipeline` wraps the actual call in `try/except` and returns a normalized `{"ok": ...}` dict — never letting a content-filter block or masking error propagate as a raw, hard-to-handle exception up to the UI layer.

File 5 — `orchestration_agent.py` (`OrchestrationAgent` class + CLI)

```

class OrchestrationAgent:
    def __init__(self, orchestration_service, mode="general", memory_window=10):
        self.orchestration_service, self.mode = orchestration_service, mode
        self.buffer = ConversationBuffer(max_turns=memory_window)

    def _build_config(self) -> OrchestrationConfig:
        use_grounding, use_safety = self.mode == "grounded", self.mode == "strict"
        # ... builds template/llm/masking/filtering/grounding based on self.mode, same
        ↪ pattern as build_full_config

    def ask(self, user_text: str) -> str:
        config = self._build_config()
        try:
            result = self.orchestration_service.run(config=config,
                template_values=[TemplateValue(name="text", value=user_text)],
    ↪ messages_history=self.buffer.history)
            answer = result.orchestration_result.choices[0].message.content
            self.buffer.add_user(user_text); self.buffer.add_assistant(answer)
            return answer
        except Exception as e:
            return f"Request blocked or failed ({self.mode} mode): {e}"

```

```
def set_mode(self, mode: str):
    self.mode = mode
    self.buffer.reset() if hasattr(self.buffer, "reset") else setattr(self, "buffer",
        ↳ ConversationBuffer()) # fresh convo on mode switch
```

Working explanation: switching mode (general/grounded/strict) resets the conversation buffer deliberately — each mode uses a differently-SHAPED template (grounded mode’s template expects a groundingOutput placeholder that general mode’s template never fills), so carrying history built under one template into a different template risks placeholder mismatches. This class is the orchestration-world equivalent of sheet 06’s SAPChatbot — same “config-driven, mode-switchable, memory-carrying agent behind a .ask()/chat() call” shape, reused a third time.

Key Points

- Managed grounding (this sheet) vs. hand-rolled RAG (sheet 05): same underlying idea, different execution location (platform-side vs. app-side Python).
- strict: true JSON schema output constrains generation itself — structurally safer than a prompt instruction to “respond in JSON.”
- Orchestration has **no server-side session memory** — messages_history must be rebuilt and resent by the client every call, exactly like sheet 05’s ChatMemory.
- run_pipeline/.ask() both catch exceptions and return a structured “blocked/failed” result instead of raising — content filter blocks are an EXPECTED outcome for a chat app, not a bug.
- The “config-driven agent class with a .ask() method and swappable mode” pattern now appears three times across this pack: QueryAgent (03), RAGAgent/SAPChatbot (05/06), OrchestrationAgent (08).

Interview Q&A

Q: How does grounding wire retrieved context into the prompt, mechanically? A: Via two linked placeholder names — input_params tells the grounding module which placeholder holds the search query; output_param is where it writes results back; your Template’s UserMessage must reference both placeholders explicitly.

Q: Why is strict JSON schema output more reliable than a prompt asking for JSON?

A: A prompt instruction is advisory — the model can still drift (extra prose, wrong types). response_format with strict: true constrains the actual token generation process, so the output is guaranteed to structurally validate — no defensive regex/parsing cleanup needed downstream.

Q: What’s the risk of NOT capping ConversationBuffer’s history length? A: Token cost (and latency) grows every turn since the full history is resent on each call — an unbounded buffer eventually hits context-window limits and/or becomes needlessly expensive; a sliding window trades “perfect memory” for bounded, predictable cost.

Q: Why does OrchestrationAgent.set_mode() reset conversation history instead of preserving it across mode switches? A: Each mode’s Template has a different message shape/placeholder set (e.g. grounded mode expects groundingOutput). Carrying history collected under one template’s shape into another risks broken placeholder substitution — resetting is the safer, simpler default than trying to reconcile incompatible histories.